Deep Panchal

TY9-B-38

DWM_Experiment no_07

AIM: Implementation of Data Discretization & Visualization using Python codes

Algorithm

1. Import required Python libraries (pandas, numpy, matplotlib, seaborn).
2. Generate or load continuous numeric data (e.g., Age).
3. Apply Equal Width Discretization using pd.cut.
4. Apply Equal Frequency (Quantile) Discretization using pd.qcut.
5. Apply Custom Binning using predefined bin ranges and labels.
6. Visualize:

- Original continuous data using a histogram.
- Discretized data using count plots.

7. Discretized data using count plots.
8. Interpret and analyze the results based on visualizations.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# For consistent plot styles
sns.set(style="whitegrid")



# Set random seed for reproducibility
np.random.seed(42)

# Generate synthetic 'Age' data from a normal distribution
data = pd.DataFrame({
    'Age': np.random.normal(loc=35, scale=10, size=1000)
})

# Clip ages to a realistic range (e.g., 18 to 70 years)
data['Age'] = data['Age'].clip(lower=18, upper=70)

# Display summary statistics for clarity
print("Data Summary:")
```

```
print(data['Age'].describe())
```

```
Data Summary:
count    1000.000000
mean       35.316459
std         9.507432
min        18.000000
25%        28.524097
50%        35.253006
75%        41.479439
max        70.000000
Name: Age, dtype: float64
```

```
# Divide the age range into 5 equal intervals
data['Age_EqualWidth'] = pd.cut(data['Age'], bins=5)

# Examine the first few rows to see the bins
print("Equal Width Bins:")
print(data[['Age', 'Age_EqualWidth']].head())
```

```
Equal Width Bins:
         Age Age_EqualWidth
0  39.967142   (38.8, 49.2]
1  33.617357   (28.4, 38.8]
2  41.476885   (38.8, 49.2]
3  50.230299   (49.2, 59.6]
4  32.658466   (28.4, 38.8]
```

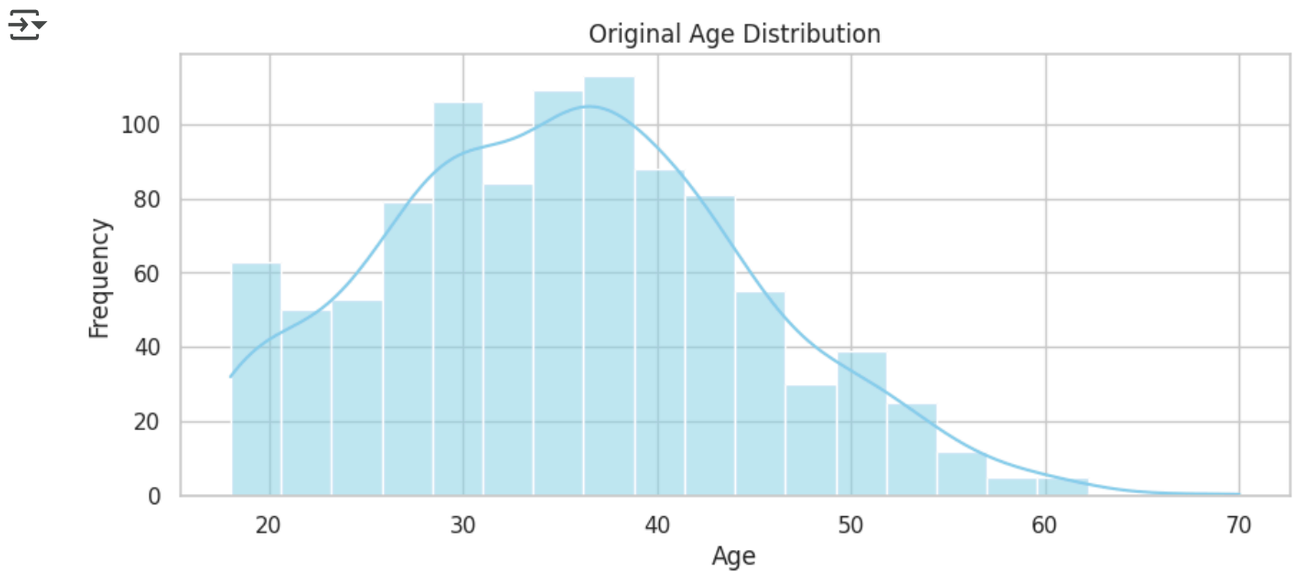
```
# Discretize into 5 bins so that each bin has a roughly equal count
data['Age_Quantile'] = pd.qcut(data['Age'], q=5)

# Display bins
print("Equal Frequency Bins:")
print(data[['Age', 'Age_Quantile']].head())
```

```
Equal Frequency Bins:
         Age      Age_Quantile
0  39.967142  (37.487, 43.135]
1  33.617357  (32.593, 37.487]
2  41.476885  (37.487, 43.135]
3  50.230299    (43.135, 70.0]
4  32.658466  (32.593, 37.487]
```

```
# Define custom bin edges and labels. Adjust the boundaries based on domain knowledge.
bins = [18, 25, 35, 50, 70]
labels = ['18-25', '26-35', '36-50', '51-70']

# Create bins using the custom boundaries
data['Age_Custom'] = pd.cut(data['Age'], bins=bins, labels=labels, include_lowest=True)

# Check the custom bins
print("Custom Bins:")
print(data[['Age', 'Age_Custom']].head())
```

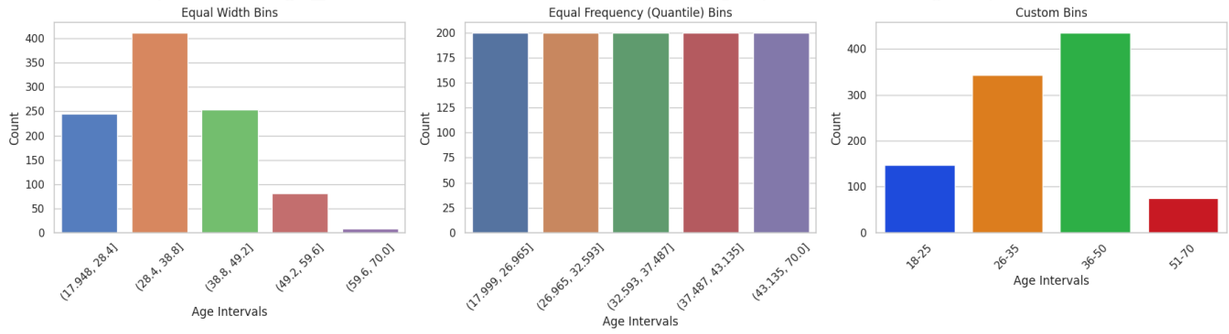
```
Custom Bins:
         Age Age_Custom
0  39.967142      36-50
1  33.617357      26-35
2  41.476885      36-50
3  50.230299      51-70
4  32.658466      26-35
```

```
plt.figure(figsize=(10, 4))
sns.histplot(data['Age'], bins=20, kde=True, color='skyblue')
plt.title('Original Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```




```
fig, axs = plt.subplots(1, 3, figsize=(18, 5))

# Equal Width Bins visualization
sns.countplot(x='Age_EqualWidth', data=data, ax=axs[0], palette='muted')
axs[0].set_title('Equal Width Bins')
axs[0].set_xlabel('Age Intervals')
axs[0].set_ylabel('Count')
axs[0].tick_params(axis='x', rotation=45)

# Equal Frequency (Quantile) Bins visualization
sns.countplot(x='Age_Quantile', data=data, ax=axs[1], palette='deep')
axs[1].set_title('Equal Frequency (Quantile) Bins')
axs[1].set_xlabel('Age Intervals')
axs[1].set_ylabel('Count')
axs[1].tick_params(axis='x', rotation=45)

# Custom Bins visualization
sns.countplot(x='Age_Custom', data=data, ax=axs[2], palette='bright')
axs[2].set_title('Custom Bins')
axs[2].set_xlabel('Age Intervals')
axs[2].set_ylabel('Count')
axs[2].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

```
<ipython-input-20-2ca8a09cfdcb>:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.

  sns.countplot(x='Age_EqualWidth', data=data, ax=axs[0], palette='muted')
<ipython-input-20-2ca8a09cfdcb>:11: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.

  sns.countplot(x='Age_Quantile', data=data, ax=axs[1], palette='deep')
<ipython-input-20-2ca8a09cfdcb>:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.

  sns.countplot(x='Age_Custom', data=data, ax=axs[2], palette='bright')
```



Review Questions

Q1:How does the process of Equal Width Discretization transform continuous data into discrete categories, and what role does the choice of the number of bins play in the outcome?

Answer:

Equal Width Discretization divides the entire range of a continuous variable into intervals of equal size. Each data point is then assigned to a bin based on its value. The choice of number of bins (k) plays a critical role:

Too few bins can oversimplify the data and hide patterns.

Too many bins can cause over-segmentation and may highlight noise.

Bin width is calculated as:

$$\text{Width} = \frac{\max - \min}{k}$$

Q2:What insights can be derived from the histogram of the original continuous data, and how does it help in understanding the distribution of the data after discretization?

Answer:

The histogram of the original data reveals:

Central tendency (e.g., average age).

Spread (range, standard deviation).

Skewness or symmetry of the distribution.

Presence of outliers.

Q3:What are the potential advantages and limitations of using Equal Width Discretization for continuous data, and how can the choice of bin width affect the analysis of the data?

Answer:

Advantages:

1. Simple to implement.
2. Easy to interpret.
3. Suitable when data is evenly distributed.

Limitations:

1. Doesn’t account for the distribution of data; may create bins with very few or too many entries.
2. Poor for skewed or multimodal data.
3. Sensitive to outliers.

Effect of Bin Width:

1. Narrow bins increase granularity but can fragment data.
2. Wide bins lose detail and may hide meaningful patterns.
3. Choice of bin width should be informed by data distribution and analysis goals.

CONCLUSION:

Data discretization was implemented using Equal Width, Equal Frequency, and Custom Binning methods. Visualization through histograms and count plots helped compare how each method groups continuous data. Discretization simplifies data analysis, improves interpretability, and is useful for preparing data for machine learning models.

GITBUB: https://github.com/panchaldeep1123/dwm